

Build a complete production-ready User Registration module for my Spring Boot e-commerce backend.

Implement a REST API endpoint: POST /api/auth/register.

The registration form should accept:

- Full Name
- Email Address
- Mobile Number
- Password
- Confirm Password

Requirements:

1. Validate every field using Bean Validation and custom validations.

2. Full Name:

- Required
- 3–100 characters
- Only alphabets and spaces.

3. Email:

- Required
- Valid email format
- Convert to lowercase

- Must be unique.

4. Mobile Number:

- Required
- Exactly 10 digits
- Numbers only
- Must be unique.

5. Password:

- Required
- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character.

6. Confirm Password must match Password.

Registration flow:

- Validate the request.
- Check whether the email already exists.
- Check whether the mobile number already exists.
- Encrypt the password using BCryptPasswordEncoder.

- Save the user in MySQL.
- Generate a JWT Access Token and Refresh Token immediately after successful registration.
- Return the user details (excluding password) along with both JWT tokens.

JWT Requirements:

- Use Spring Security with JWT authentication.
- Generate Access Token (15 minutes).
- Generate Refresh Token (7 days).
- JWT should contain userId, email, and role as claims.
- Configure the project for stateless authentication.

Create the following:

- User Entity
- RegistrationRequest DTO
- RegistrationResponse DTO
- UserRepository
- AuthService
- AuthController

- JwtService
- JwtAuthenticationFilter
- SecurityConfig
- Password encoder configuration
- Global Exception Handler (@ControllerAdvice)
- Custom exceptions for duplicate email, duplicate mobile number, invalid password, and password mismatch.

Database Requirements:

- Email must be UNIQUE.
- Mobile Number must be UNIQUE.
- Store passwords only in encrypted form.
- Add createdAt and updatedAt timestamps.

Return standardized JSON responses for both success and failure.

Follow Spring Boot best practices, SOLID principles, clean architecture, and production-ready coding standards. Generate all required classes, configurations, and code without placeholders.